

Time Series Mining & Forecasting



Dr. Thanuja Ambegoda

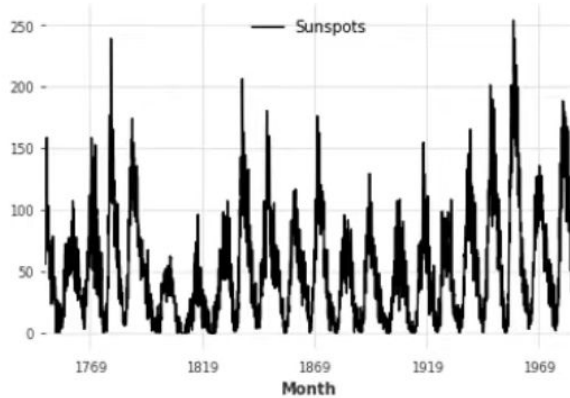
Lecture outline

1. Introduction to TS
2. Basic visualization techniques
3. Trend analysis and seasonality detection
4. Cyclic pattern analysis
5. Anomaly detection in TS
6. Stationarity in TS
7. Traditional TS forecasting methods
8. Advanced TS forecasting methods

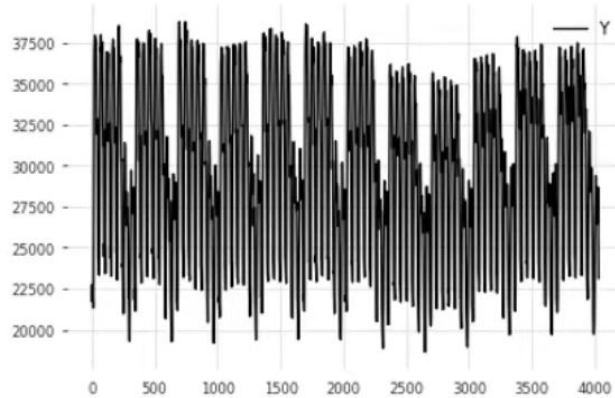
Part 1.

Introduction to time series analysis and mining

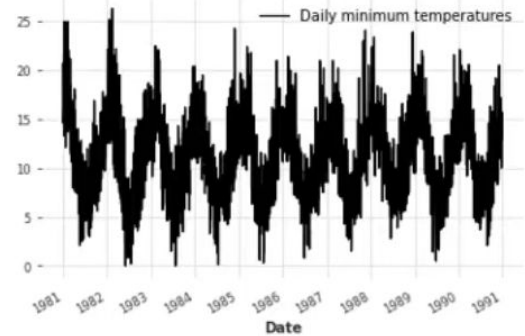
Time series everywhere!



Monthly Sunspot Numbers, 1749 - 1983



Half-hourly electricity demand in England and Wales

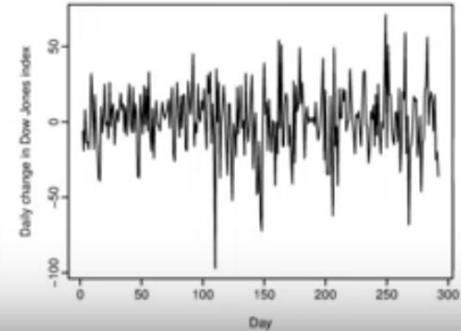
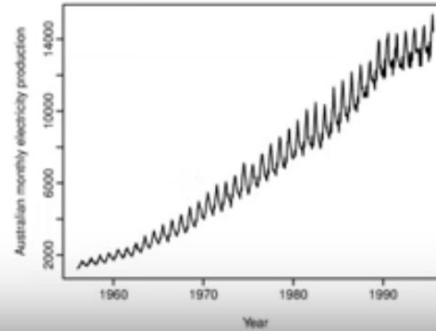
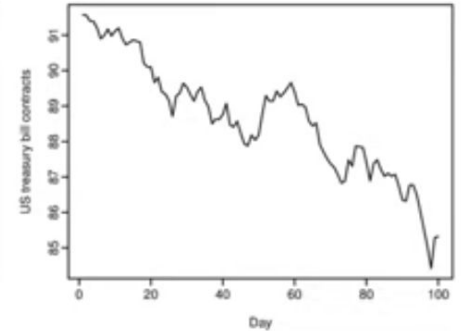
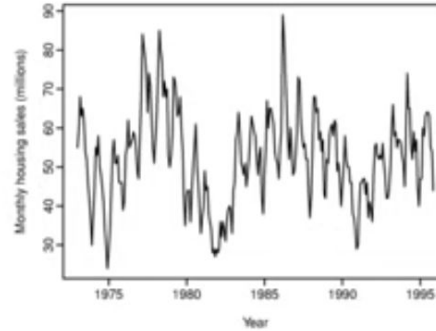


Daily temperature in Melbourne between 1981 and 1990



What is a time series?

- A **time series** is an ordered sequence of values of a variable usually at equally spaced time intervals
- Typically, the mean and the standard deviation can vary in a time series
- Understanding time series data helps predict future events based on historical patterns



Trend

Volatility

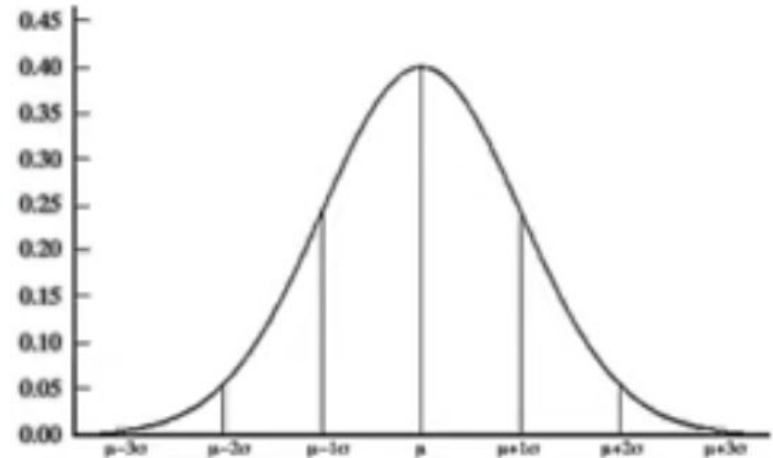
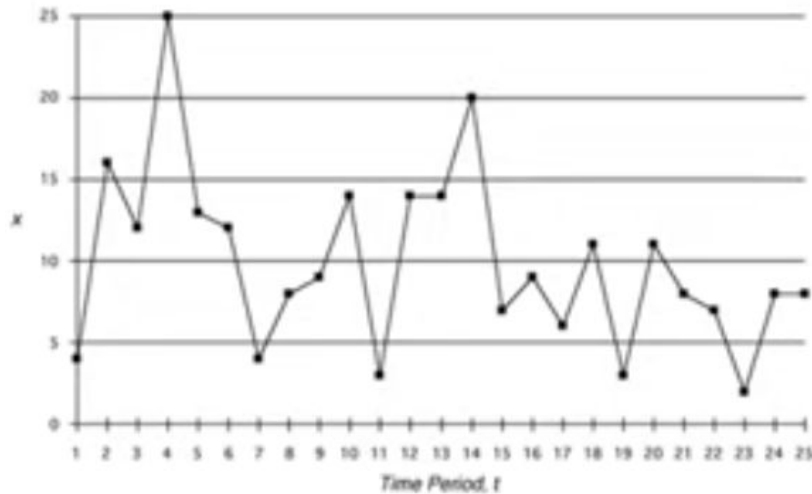
TS analysis, forecasting & mining

- Time series analysis
 - Techniques for identifying and analysing temporal dependencies in sequential data
- Time series mining
 - What has happened?
- Time series forecasting
 - What's going to happen next?

Applications

- Finance
 - Sales forecasting
 - Inventory analysis
 - Stock market analysis
 - Price estimation
- Weather
 - Temperature
 - Climate change
 - Seasonal shifts
 - Rain, wind, snow,...
- Industry
 - Usage
 - Anomaly
 - Maintenance
- Healthcare
 - Patient occupancy in hospitals
 - Disease outbreaks
 - Patient monitoring
- Insurance
 - Sales forecasting
 - Insurance benefits awarded

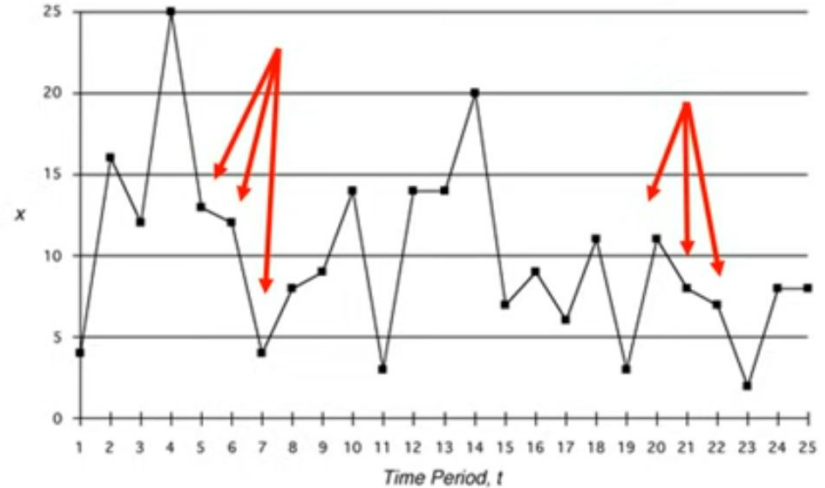
Difference between a time series and a random variable with known mean and standard deviation



We might be able to build a distribution that explains all the data points in the time series. But does that mean the distribution can be sampled to produce the time series?

Dependence on the past

- **Memory** of a TS
 - The extent to which past values influence future values
 - A few points having a similar trend
 - But there might be no overall trend
- Actual values tend to depend on the value at the previous time point
 - **Autocorrelation**



Memory of a Time Series

1. Short memory
 - a. Short-term dependencies. Only recent past values influence the next value
 - b. E.g. financial data
2. Long memory
 - a. Long-term dependencies. Distant past values can impact the next value.
 - b. E.g. climate data

Stationary TS often have short memory

Non-stationary TS often have long memory but require more complex models

Time Series Components

Time series components

Level: The average value in the series.

Trend: The increasing or decreasing value in the series.

Cycle: (Long-term) phenomena that happen across seasonal periods, without a fixed period (eg. economic recession)

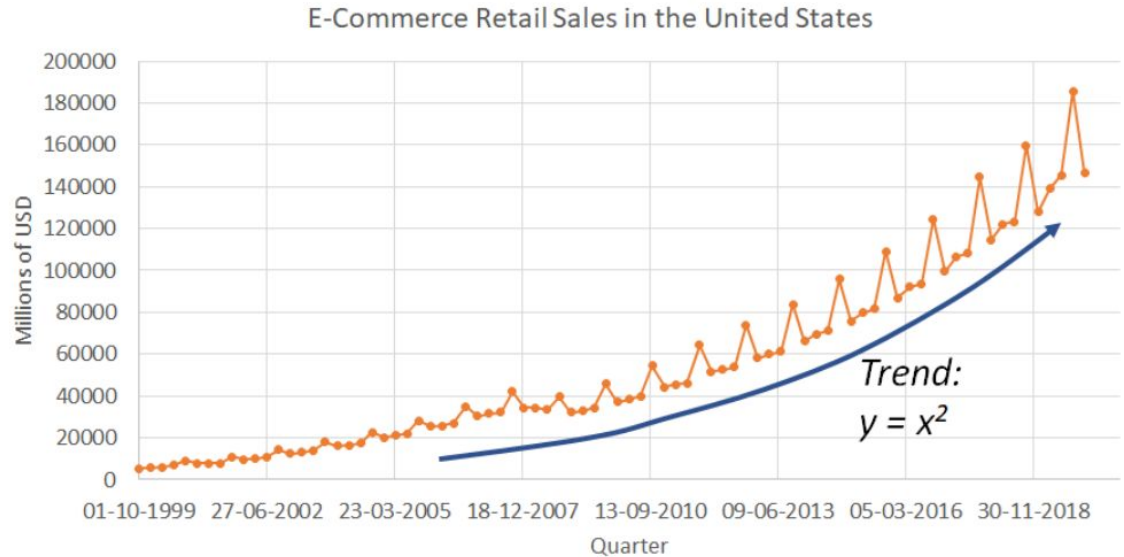
Seasonality: The repeating short-term cycle in the series.

Noise: The random variation in the series. (unsystematic)

[Reference](#)

Trend

- Refers to the increasing or decreasing value in the series.
- Can be linear or nonlinear.
- Methods to detect:
 - Moving Averages
 - Polynomial Fitting



Seasonal component



Periodic ups and downs. Period can be a year, several years, months, weeks or days.

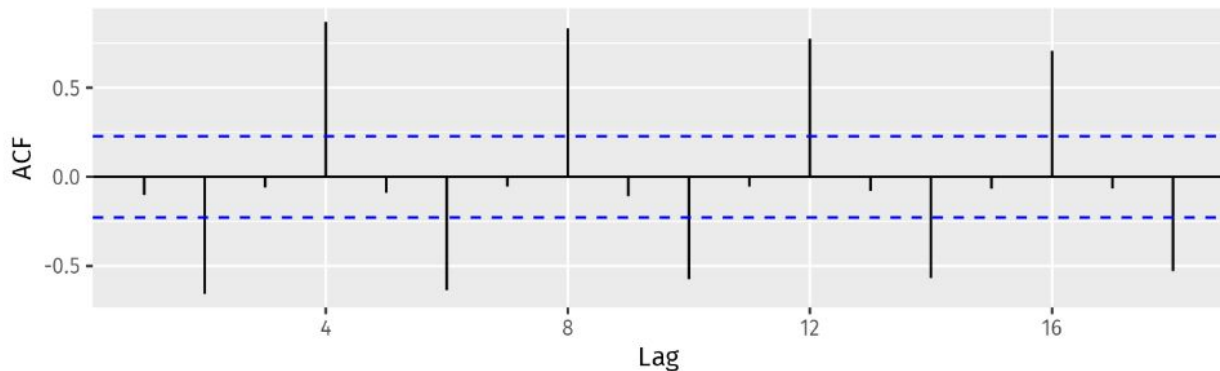
How to measure/detect seasonality

1. Autocorrelation plots
2. FFT (Fast Fourier Transform)
3. Seasonal decomposition

Autocorrelation Plots

- Autocorrelation, often denoted as $R(k)$, measures the correlation between a time series and its lagged version.
- A significant spike at a particular lag indicates seasonality at that lag.

$$r_k = \frac{\sum_{t=k+1}^T (y_t - \bar{y})(y_{t-k} - \bar{y})}{\sum_{t=1}^T (y_t - \bar{y})^2},$$



If there's seasonality present at a particular lag k , the autocorrelation plot will show a significant spike at that lag. For example, if there's a yearly seasonality in monthly data, we would expect a significant autocorrelation at lag 12.

Decomposing TS into components

Often this is done to help improve understanding of the time series, but it can also be used to improve forecast accuracy.

- Additive models (linear)
 - $y(t) = \text{Level} + \text{Trend} + \text{Seasonality} + \text{Noise}$
- Multiplicative models (nonlinear)
 - $y(t) = \text{Level} * \text{Trend} * \text{Seasonality} * \text{Noise}$

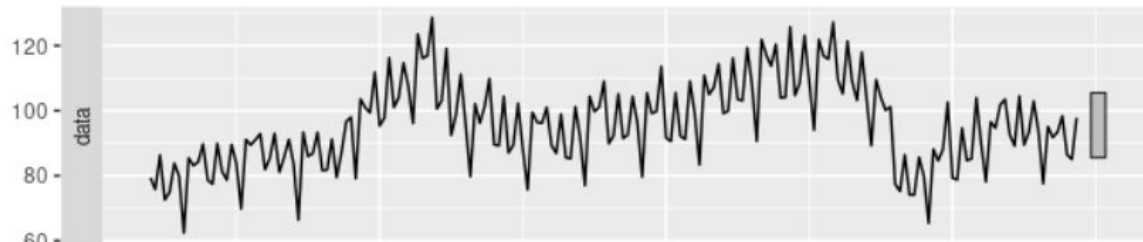
Real-world problems are messy and noisy.

- There may be additive and multiplicative components.
- There may be an increasing trend followed by a decreasing trend.
- There may be non-repeating cycles mixed in with the repeating seasonality components.

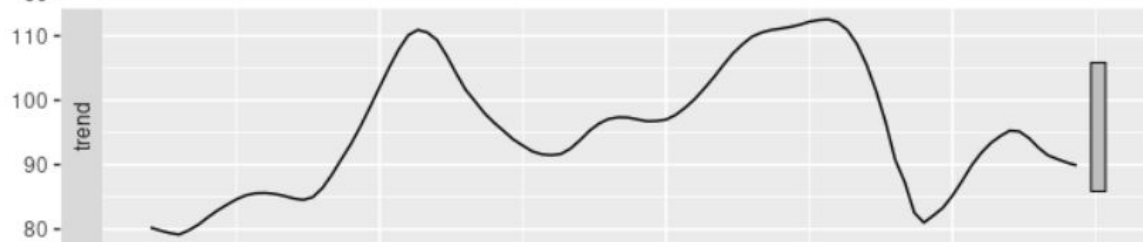
Decomposing TS into components



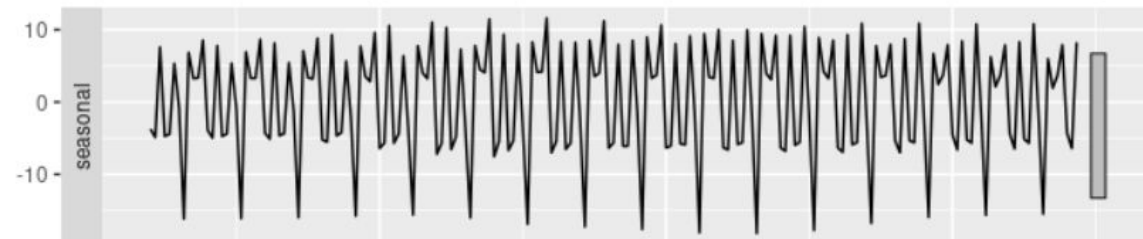
Original



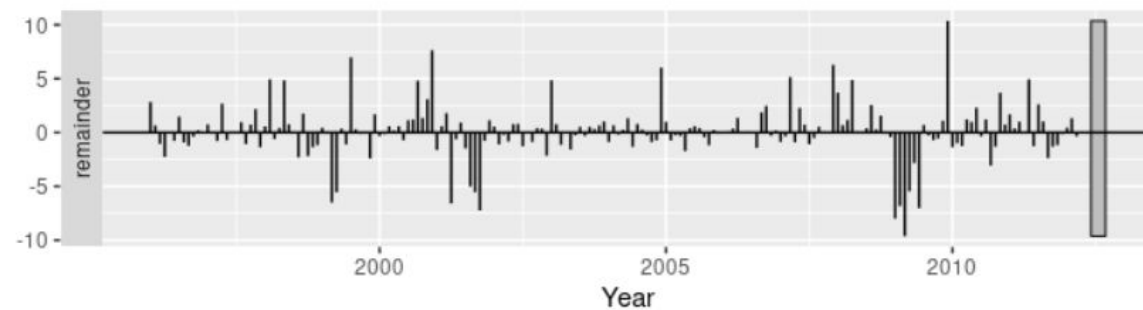
Trend-cycle
component



Seasonal
component

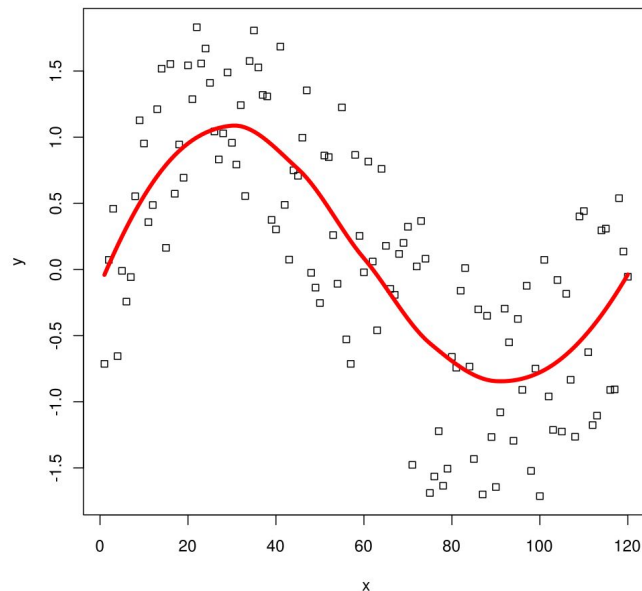


Residual
component



Seasonal and Trend Decomposition using LOESS (STL)

- **LOESS**: Locally Weighted Scatterplot Smoothing
- Alternatively **LOWESS**, which means Locally Weighted Regression
- Non-parametric smoothing technique.
- Fits smooth curves using locally weighted polynomial regression.
- Each point estimated based on subset of its neighbors.
- Close neighbors have more influence via weighted least squares.
- Ensures gradual transitions, yielding a smooth curve.

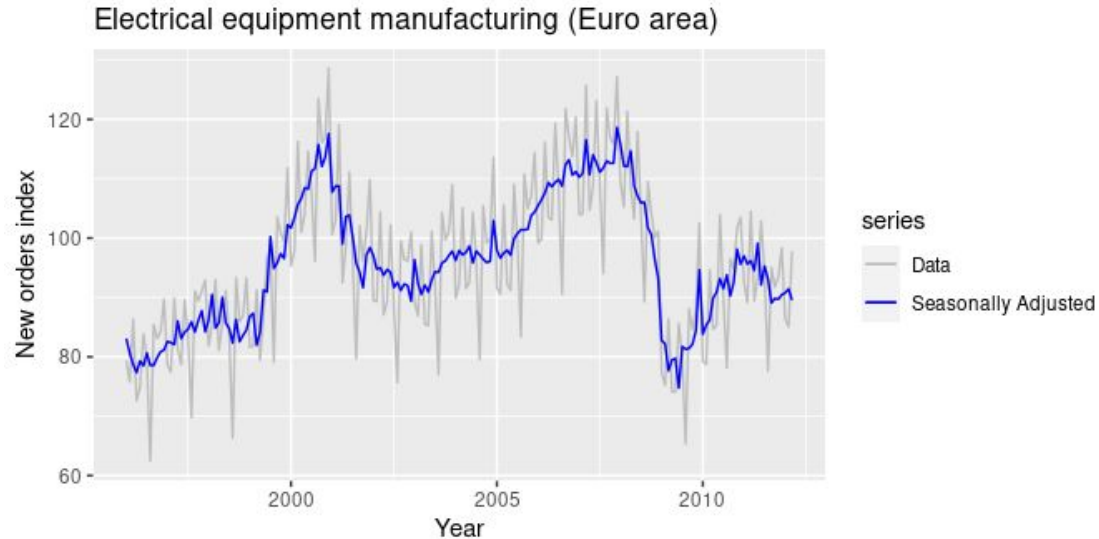


Steps in STL

1. Trend extraction
 - a. Applies LOESS smoother to time series.
 - b. Captures the central path or growth trajectory.
 - c. Not influenced by seasonality.
2. Detrending & Seasonal Extraction
 - a. Remove trend from original series.
 - b. Apply LOESS to detrended series to capture seasonality.
 - c. Seasonal pattern can be of any type: monthly, quarterly, etc.
3. Extract Residual Component
 - a. $\text{Residuals} = \text{Original Series} - \text{Trend} - \text{Seasonality}$.
 - b. Captures randomness and potential anomalies.
 - c. Helpful for model diagnostics.

Seasonally adjusted data

If the variation due to seasonality is not of primary interest, the seasonally adjusted series can be useful. For example, monthly unemployment data are usually seasonally adjusted in order to highlight variation due to the underlying state of the economy rather than the seasonal variation.

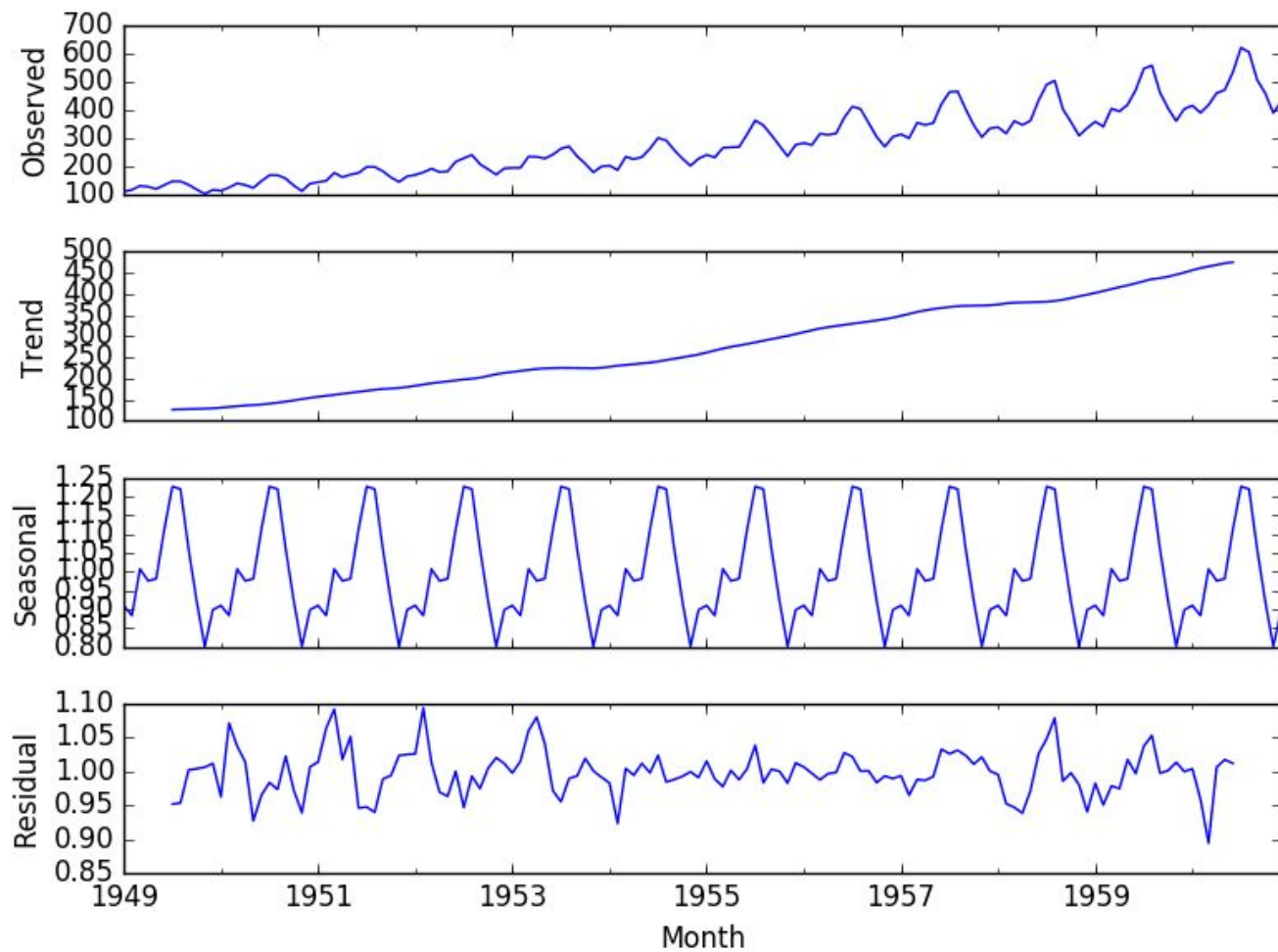


TS decomposition using statsmodel package

Naive, or classical, decomposition method. Requires you to know if the model is additive or multiplicative

```
1 from random import randrange
2 from pandas import Series
3 from matplotlib import pyplot
4 from statsmodels.tsa.seasonal import seasonal_decompose
5 series = [i+randrange(10) for i in range(1,100)]
6 result = seasonal_decompose(series, model='additive', period=1)
7 result.plot()
8 pyplot.show()
```

```
1 from pandas import read_csv
2 from matplotlib import pyplot
3 from statsmodels.tsa.seasonal import seasonal_decompose
4 series = read_csv('airline-passengers.csv', header=0, index_col=0)
5 result = seasonal_decompose(series, model='multiplicative')
6 result.plot()
7 pyplot.show()
```



Cycles

- Oscillations that aren't fixed in a seasonal pattern.
- Typically longer duration than seasonality.
- Can be caused by economic conditions, large-scale events, etc.

Why detect cycles?

- Improve forecasting accuracy. Uncover underlying long-term patterns.
- Make informed strategic decisions (e.g., economic, business planning).
- Separate cyclical effects from irregular noise.

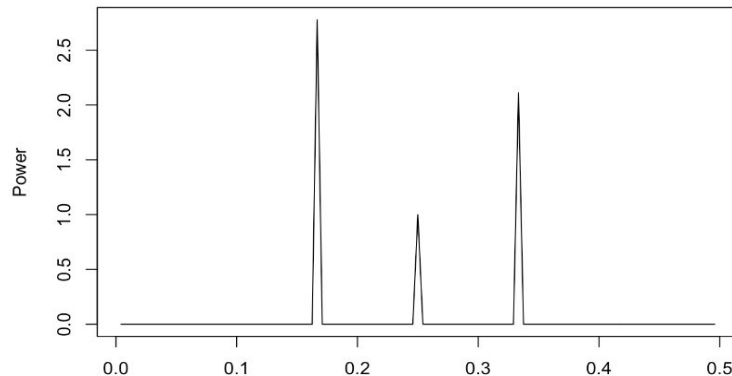
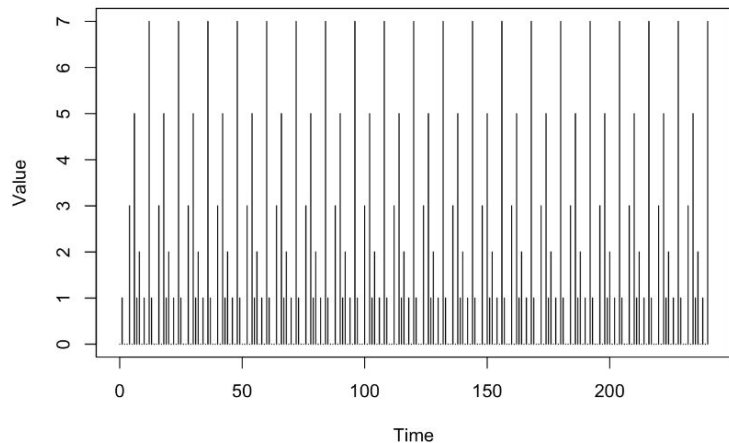
Detecting cycles

1. Spectral analysis

- Transform time series into frequency domain using Fourier Transform.
- Identify dominant frequencies (apart from seasonal frequencies).
- The peaks in spectral density indicate cyclical components.

Seasonality shows up as distinct peaks at regular intervals in the spectral density plot.

Cyclical components will manifest as broader peaks or bands at lower frequencies, representing longer-term fluctuations.



Detecting cycles

2. Wavelet transform

- Decompose time series into different frequency bands.
- Detect cycles of various lengths.
- Suitable for non-stationary time series

Seasonality: Seasonal components will appear as consistent patterns at specific scales (frequencies) across the time domain.

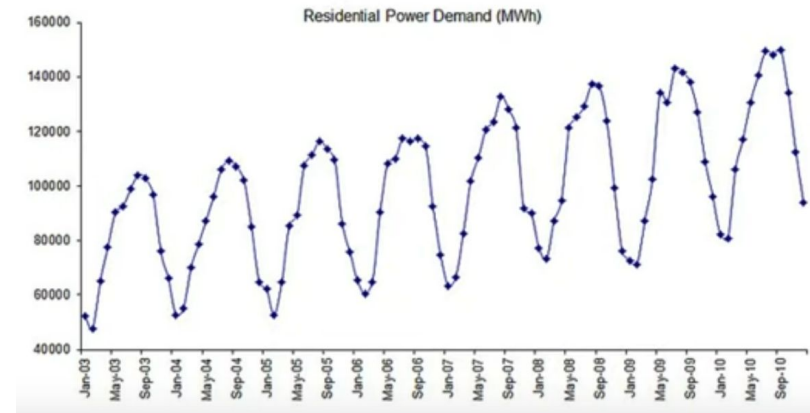
Cycles: Cyclical components will appear as patterns at broader scales that may vary over time.

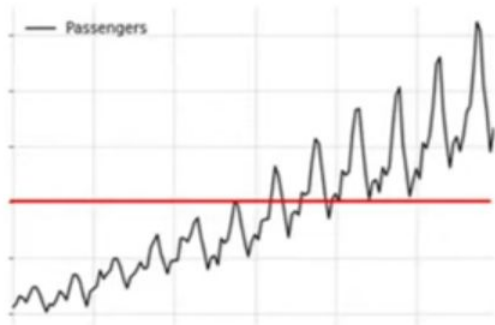
TS and regression

- Focused on identifying underlying trends and patterns
- Mathematically modeling / describing those patterns
- Predict/forecast future values

TS and regression

- Regression can model the trend over time
 - Linear regression for mean demand. Function of economic growth?
 - Within year cycles - seasonal variability?
 - $Y = b_1x_1 + b_2x_2 + b_3x_3 + \dots$
- TS allows you to model the process without knowing the underlying causes

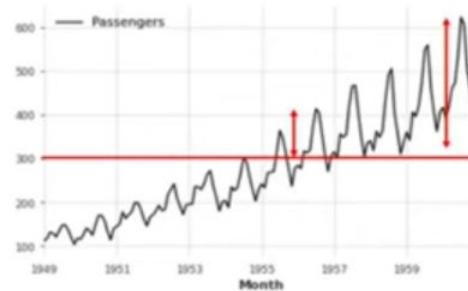




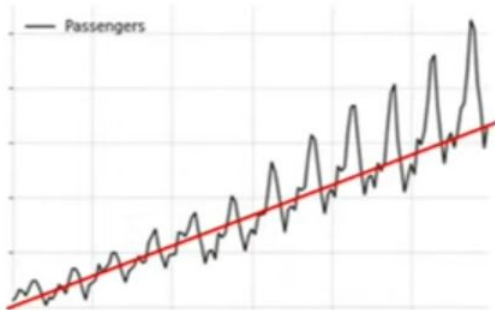
Level



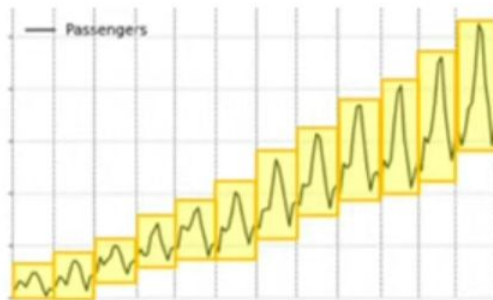
Noise



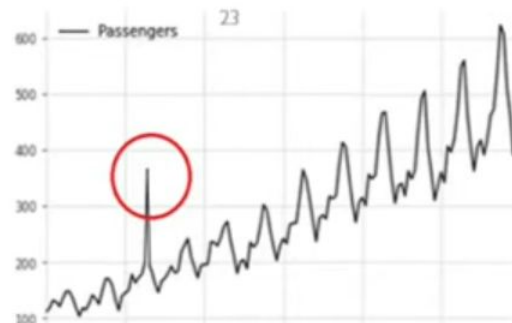
Non-constant Variance



Trend



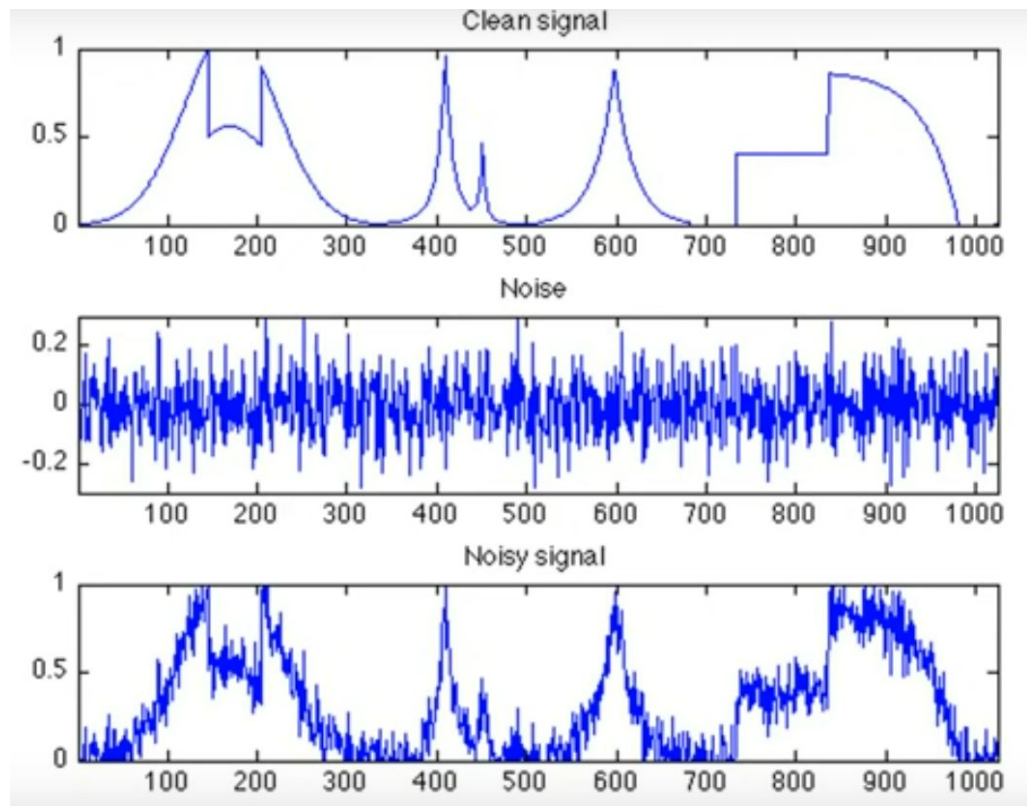
Seasonality



Outlier

Signal and Noise

Signal and noise



Signal and noise: some definitions

- Statistical moments
 - Mean and standard deviation
- Stationary vs non-stationary
 - Trends in mean and/or standard deviation
 - Stationary - doesn't depend on the time of observation
- Seasonality
 - Periodic patterns
- Autocorrelation
 - Degree to which the time series values in period t are related to time series values in period $t-1$, $t-2$, ...

Stationarity of a Time Series

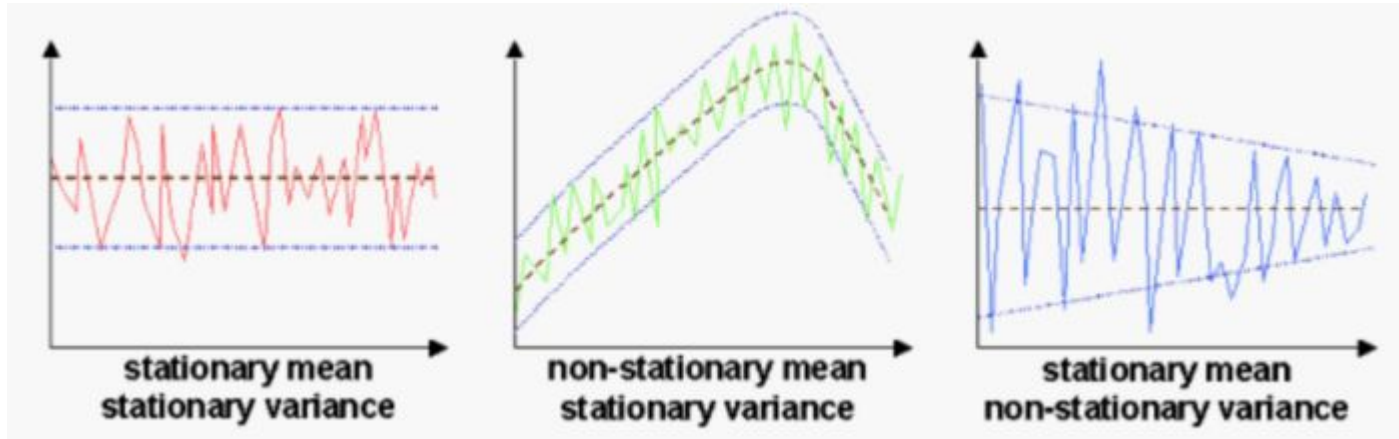
Stationary TS

TS whose properties do not depend on the time of observation

Constant variance and level

No seasonality

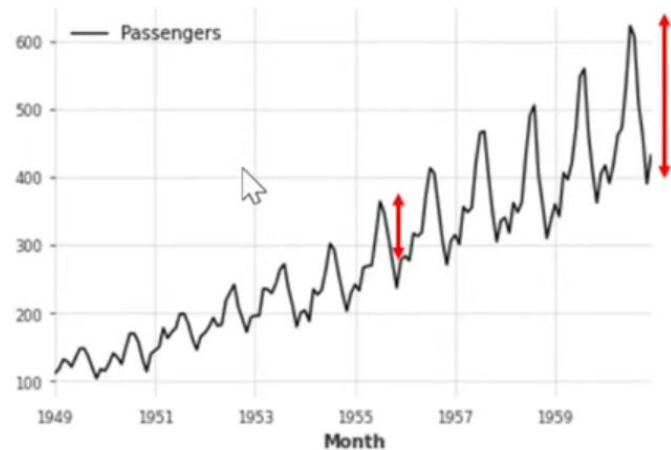
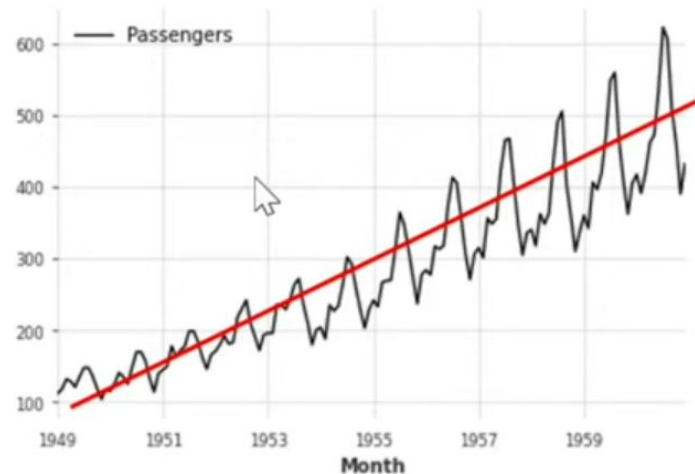
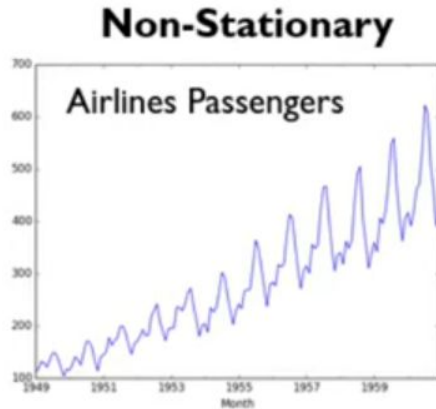
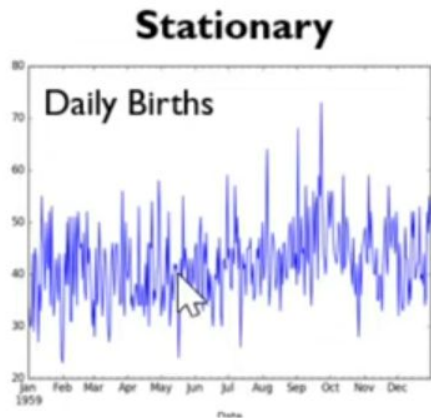
No trend



Stationarity check

1. Visual inspection
2. Seasonal-Trend decomposition
3. Summary statistics at random partitions
4. Statistical tests

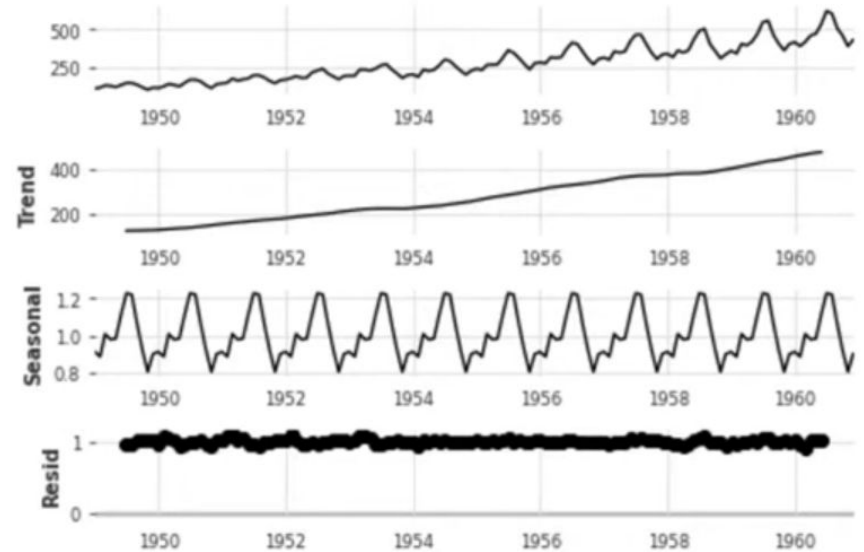
Stationarity check: Visual inspection



Mean
Variance
Seasonality

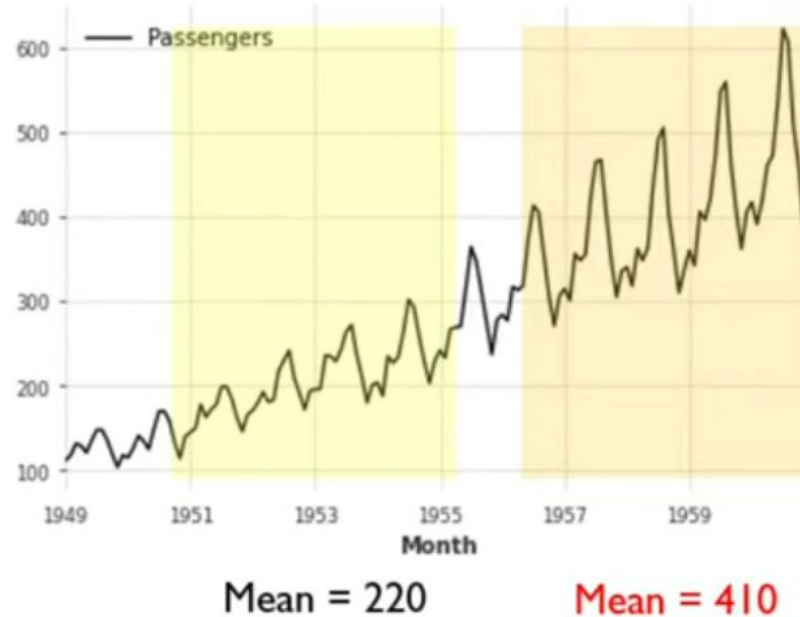
Stationarity check: Seasonal-Trend decomposition

```
from pandas import read_csv
from statsmodels.tsa.seasonal import seasonal_decompose
df = read_csv('airline-passengers.csv', header=0)
df.Month = pd.to_datetime(df.Month)
df = df.set_index('Month')
result = seasonal_decompose(df, model='multiplicative')
result.plot()
```



Stationarity check: summary statistics

Summary statistics at random partitions



Stationarity check: statistical tests

Augmented Dickey-Fuller test

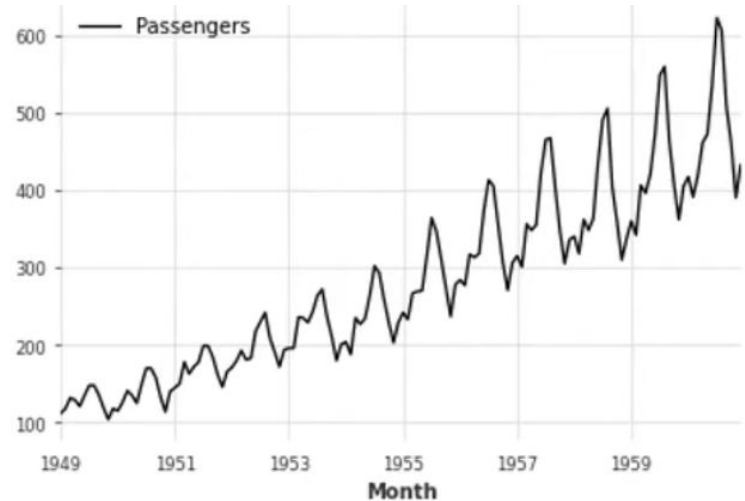
```
from pandas import read_csv
from statsmodels.tsa.stattools import adfuller
df = read_csv('airline-passengers.csv', header=0, index_col=0)
result = adfuller(df.values)
print('p-value: %f % result[1])
```

P-value: 0.991880

P-value > 0.05: Fail to reject the null hypothesis (H0)

Null Hypothesis (H0): It looks non-stationary

Alternate Hypothesis (H1): It looks stationary



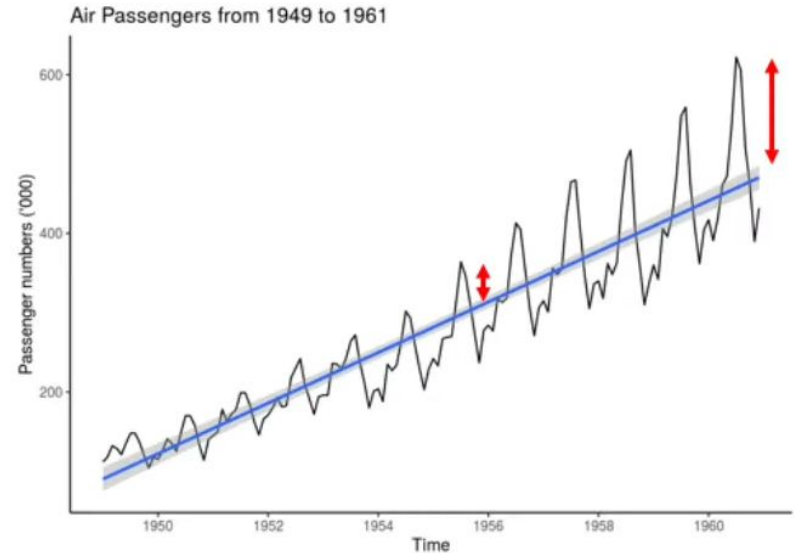
TS is non-stationary!

Adjustments

1. Fixing non-constant variance
2. Trend removal
3. Fixing non-constant variance + trend removal

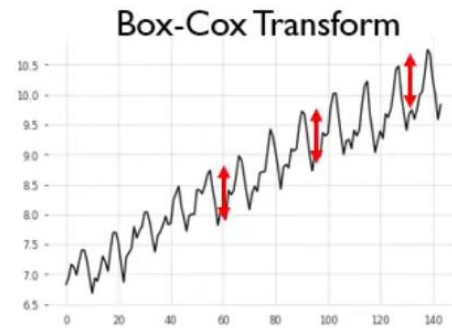
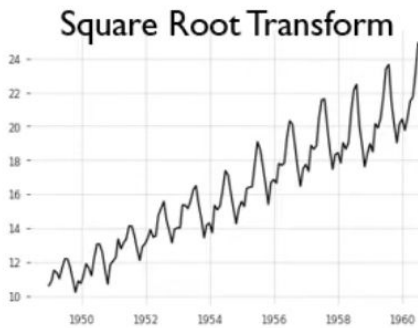
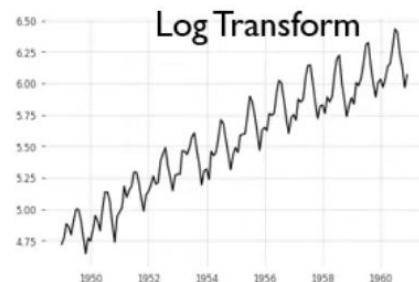
Fixing non-constant variance

- **Logarithmic**
 - Removes extreme (exponential) trends
- **Square root transformation**
 - Removes a quadratic growth trend
- **Box-Cox transform**
 - Supports both square root and log transform



Fixing non-constant variance

```
from pandas import read_csv
from numpy import log
from numpy import sqrt
from scipy.stats import boxcox
import matplotlib.pyplot as plt
series = read_csv('airline-passengers.csv', header=0,
parse_dates=[0], index_col=0, squeeze=True)
series_log = log(series)
series_sqr = sqrt(series)
series_boxcox, lam = boxcox(series)
print('Lambda: %f % lam)
plt.plot(series_log)
plt.plot(series_sqr)
plt.plot(series_boxcox)
plt.show()
```



To reverse the boxcox transform:
from scipy.special import boxcox, inv_boxcox

Trend removal

Differencing

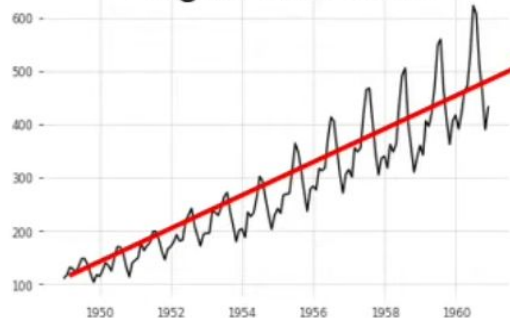
- First order : the time series value y_i is replaced by the difference between it and the previous value.

- First-order $y'_i = y_i - y_{i-1}$.

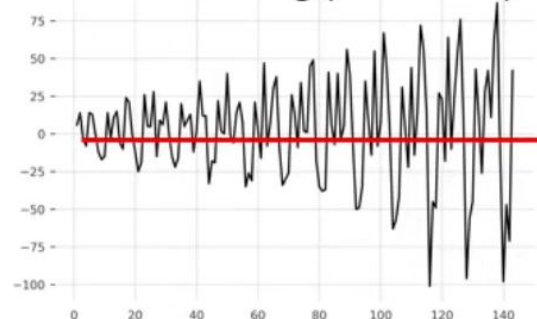
- Second-order $y''_i = y'_i - y'_{i-1}$

```
from pandas import datetime
from matplotlib import pyplot
series = read_csv('airline-passengers.csv', header=0,
parse_dates=[0], index_col=0, squeeze=True)
diff = series.diff()
pyplot.plot(series.values)
pyplot.plot(diff.values)
```

Original Time Series



After Differencing (First-order)



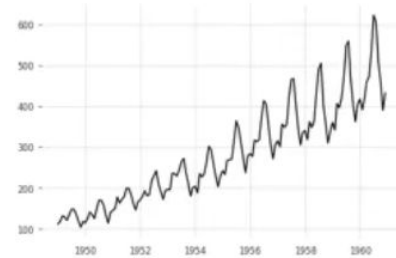
Fixing non-constant variance + trend removal

```
from pandas import read_csv
from pandas import Series
import pandas as pd
from scipy.stats import boxcox

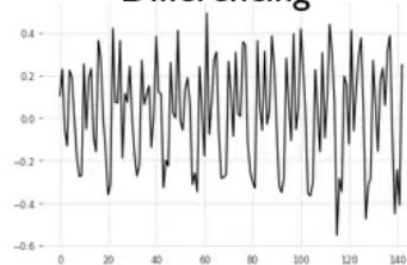
series = read_csv('airline-passengers.csv',
header=0, parse_dates=[0], index_col=0,
squeeze=True)

series_boxcox, lam = boxcox(series)
series_boxcox = pd.DataFrame(series_boxcox)
series_boxcox_diff = series_boxcox.diff()
plt.plot(series)
plt.plot(series_boxcox_diff)
```

Original Time Series



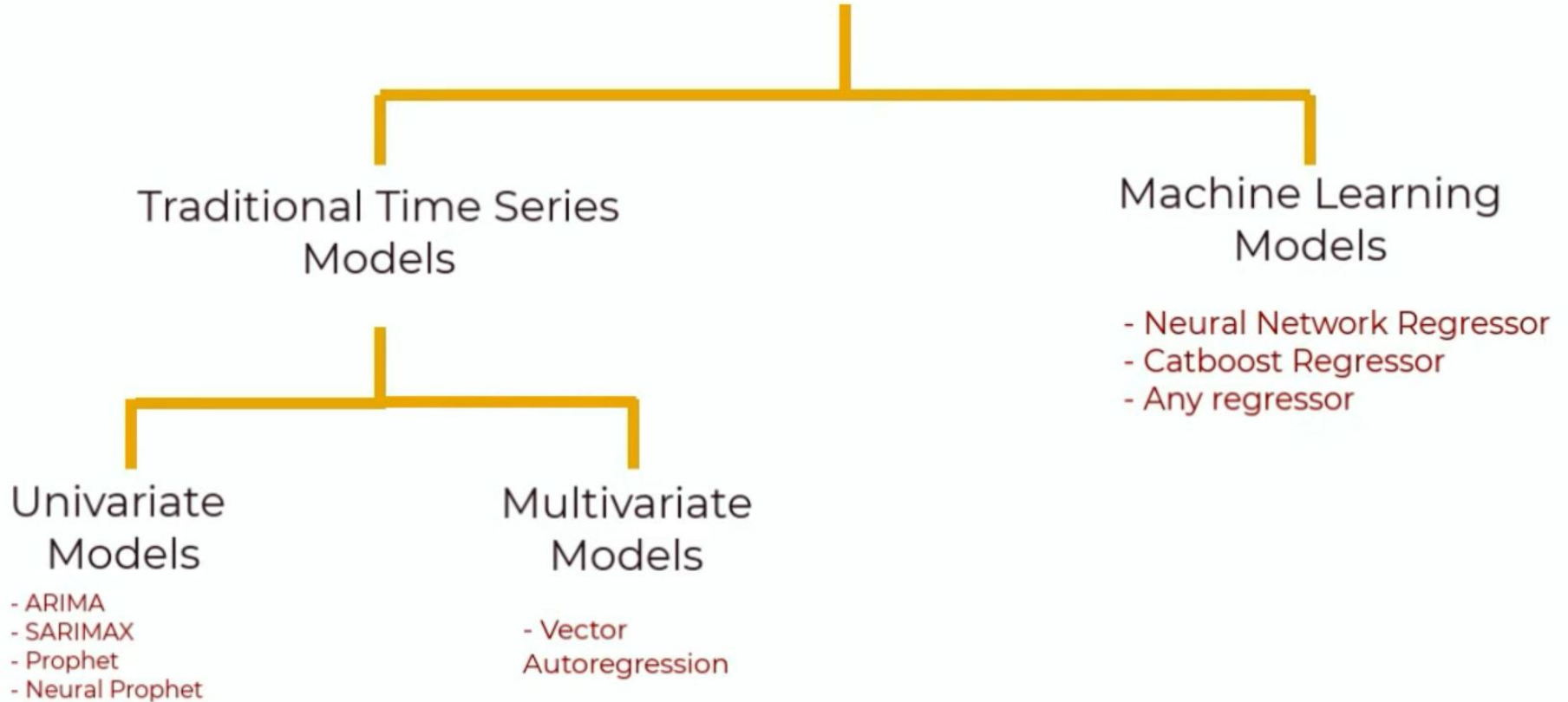
Box Cox Transform + Differencing



Part 2.

Time series forecasting

Time Series Models



Simple forecasting methods

1. Average method
2. Naive method
3. Seasonal naive method
4. Drift method

[Homework: Read about simple forecasting methods](#)

These methods are covered for the sake of completeness of fundamental theory, and can be used as primary benchmarks.

Refer to the previous lecture for better performing forecasting methods.

Time series forecasting

Estimating future values of a time series

1. Exponential smoothing
2. Autoregressive methods
 - a. AR
 - b. MA
 - c. ARMA
 - d. ARIMA
 - e. SARIMA

Exponential smoothing

Simple Exponential Smoothing (SES) (Brown, 1956) is suitable for forecasting data with no clear trend or seasonal pattern.

- Prediction is a weighted linear sum of recent past observations
- Weights decline exponentially, making observations much earlier in the past less relevant

$$\hat{y}_{T+1|T} = \alpha y_T + \alpha(1 - \alpha)y_{T-1} + \alpha(1 - \alpha)^2 y_{T-2} + \cdots,$$

where $0 \leq \alpha \leq 1$ is the smoothing parameter.

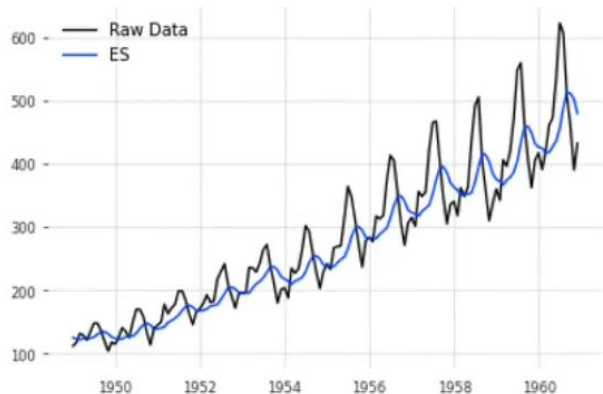
Exponential smoothing

- Double Exponential Smoothing (Holt, 1957)
 - Smoothing for level + trend
- Triple Exponential Smoothing (Holt-Winter, 1960)
 - Smoothing for trend + seasonality + level

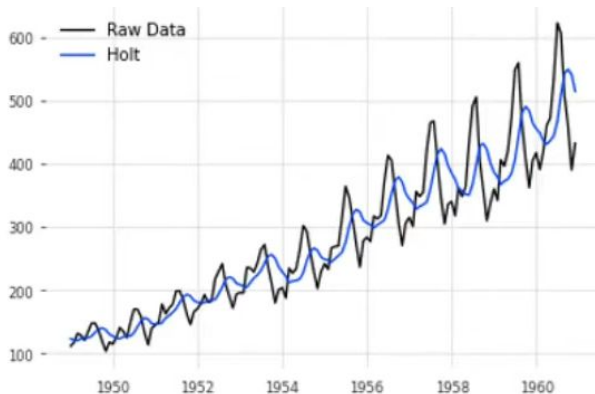
Exponential smoothing - Python

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.holtwinters import SimpleExpSmoothing
df = pd.read_csv('airline-passengers.csv', parse_dates=['Month'], index_col='Month')
df.index.freq = 'MS'
ts=df.iloc[:, 0]
es1 = SimpleExpSmoothing(ts).fit(smoothing_level=0.2)
ts_es1 = es1.predict(start=ts.index[0], end=ts.index[-1])
es2 = ExponentialSmoothing(ts, trend='add').fit(smoothing_level=0.2,smoothing_trend=0.2)
ts_es2 = es2.predict(start=ts.index[0], end=ts.index[-1])
es3 = ExponentialSmoothing(ts, trend='add',seasonal='mul',
seasonal_periods=12).fit(smoothing_level=0.2,smoothing_trend=0.2,smoothing_seasonal=0.2)
ts_es3 = es3.predict(start=ts.index[0], end=ts.index[-1])
```

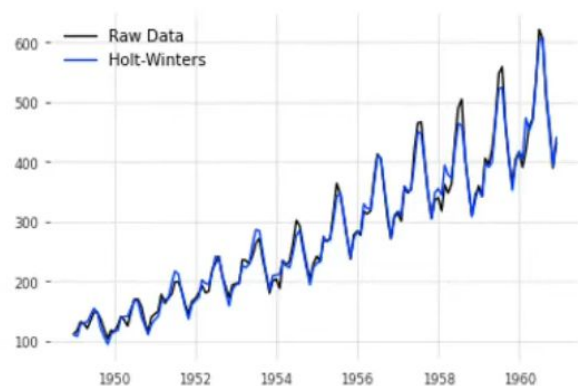
Exponential smoothing - Python



Simple Exponential Smoothing
 $\alpha=0.2$
Level only



Double Exponential Smoothing
 $\alpha=0.2, \beta=0.2$
Level+ Trend



Triple Exponential Smoothing
 $\alpha=0.2, \beta=0.2, \gamma=0.2, S=12$
Level+ Trend + Seasonality

References

- [Forecasting: Principles & Practice \(2nd edition\)](#)